Super Simple Software Factory

Installation guide for Linux, macOS and Windows

SSSF is a Claude Code *skill* that stamps an agent "factory" into a project you already have (or are starting). Installing it takes three moves: download the skill once, copy it into your project's `.claude/skills/` folder, then run its installer from the project's root folder. This guide walks through every step, from an empty machine to a first successful run.

> **Two folders, never mixed up.** The *ssaf repository* is where the skill comes from: download it once, anywhere. Your *project* is the code you want the factory to work on. The skill gets copied **from** the first **into** the second, and from step 3 on every command runs in the project's root folder.

1. Prerequisites
2. Download the skill (once)
3. Open your project's root folder
4. Copy the skill into the project
5. Stamp the factory
6. Set your API keys
7. Make the first commit
8. Wire your real test command
9. Smoke test
10. Windows notes
11. Updating an installation
12. Troubleshooting

1 Prerequisites

Tool	Needed for	Linux / macOS	Windows
Git	Required. Commit phases, change capture, the write boundary.	Your package manager (`apt install git` , `brew install git`)	**Git for Windows**, at its default location: `winget install --id Git.Git -e`
uv	Required. Runs every ADW and the installer, and fetches Python and the dependencies for you.	`curl -LsSf https://astral.sh/uv/install.sh | sh`	`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`
pi	The default coding agent (`coding_agent: pi`).	Install from `github.com/mariozechner/pi-coding-agent` , then check with `pi --version` . Not needed if every agent uses Claude Code.	
Claude Code	Only for agents set to `coding_agent: claude_code` .	`claude` on PATH, logged in, or `ANTHROPIC_API_KEY` set.	
just	Optional. The stamped recipes (`just demo` , `just obs` , …).	`brew install just` or `uv tool install rust-just`	`winget install --id Casey.Just --exact`

sqlite3	Optional. The trace-reading recipes (`just sessions` , `just phases`).	Usually preinstalled; else your package manager	Command-line tools from `sqlite.org/download.html` , with `sqlite3.exe` on PATH
bun	Optional. The trace visualizer (`just obs`).	`curl -fsSL https://bun.sh/install | bash`	`powershell -c "irm bun.sh/install.ps1 | iex"`

You also need an API key for every model provider your agent roster names (step 6).

2 Download the skill (once)

The skill lives in the `ssaf` repository on GitHub. Clone it to any folder that is **not** inside your project, for example a folder where you keep tools:

LINUX / MACOS / GIT BASH

```
mkdir -p ~/tools
cd ~/tools
git clone https://github.com/PetrosChol/ssaf.git
```

WINDOWS POWERSHELL

```
New-Item -ItemType Directory -Force C:\tools
cd C:\tools
git clone https://github.com/PetrosChol/ssaf.git
```

The skill itself is the folder `ssaf/.claude/skills/sssf` inside that clone. The rest of this guide calls the clone `~/tools/ssaf` (Windows: `C:\tools\ssaf`); use your own path if you chose another one. Note that `.claude` starts with a dot, so file browsers may hide it.

3 Open your project's root folder

The **root folder** of a project is its top-level folder: the one that holds its `.git` folder and files such as `README.md` , `pyproject.toml` or `package.json` . Open a terminal there (in VS Code: *Terminal → New Terminal* opens one in the root of the open folder).

AN EXISTING PROJECT (LINUX / MACOS / GIT BASH, THEN WINDOWS POWERSHELL)

```
cd ~/projects/my-app
cd D:\projects\my-app
git status
```

A NEW, EMPTY PROJECT (LINUX / MACOS / GIT BASH)

```
mkdir my-app
cd my-app
git init
```

A NEW, EMPTY PROJECT (WINDOWS POWERSHELL)

```
New-Item -ItemType Directory my-app
cd my-app
git init
```

Use the `cd` line for your system, with your own project path. If `git status` says the folder is not a git repository, run `git init` there. If it shows you are inside a repository but in a subfolder, `git rev-parse --show-toplevel` prints the root; go there.

4 Copy the skill into the project

Still in the project's root folder, create `.claude/skills/` and copy the skill into it:

LINUX / MACOS / GIT BASH

```
mkdir -p .claude/skills
cp -r ~/tools/ssaf/.claude/skills/sssf .claude/skills/
```

WINDOWS POWERSHELL

```
New-Item -ItemType Directory -Force .claude\skills
Copy-Item -Recurse C:\tools\ssaf\.claude\skills\sssf .claude\skills\
```

Your project must now look like this, with `.claude` **next to** `.git`:

```
my-app/                    <- project root: run every command from here
├── .git/
├── .claude/
│   └── skills/
│       └── sssf/          <- the copied skill (SKILL.md, scripts/, templates/, …)
├── README.md
└── …your code…
```

Check it: `ls .claude/skills/sssf/SKILL.md` (PowerShell: `Test-Path .claude\skills\sssf\SKILL.md` must print `True`). Places that will not work: inside the `ssaf` clone, in a subfolder of the project, or one level too deep, `.claude/skills/sssf/sssf/`, which happens if the destination you copy to already ends in `sssf`. Copy to `.claude/skills/`, exactly as above.

> **Using Claude Code?** Claude Code finds skills in the project's `.claude/skills/` folder. Open Claude Code in the project root and type `/sssf install`: the agent performs steps 5 to 9 for you. The manual steps below are what it does.

Alternative: one copy for all your projects. You can put the skill in your user folder instead, at `~/.claude/skills/sssf` (Windows: `%USERPROFILE%\.claude\skills\sssf`). Claude Code then offers it in every project, and in step 5 you run the installer from that path: `uv run ~/.claude/skills/sssf/scripts/install.py`. The installer still writes into the folder you run it from, so run it from each project's root.

5 Stamp the factory

```
uv run .claude/skills/sssf/scripts/install.py
```

Run it from the project root; the same command works in every shell. It writes into the folder you run it from, which is why step 3 matters. It creates:

Path	What it is
`adws/adw_*.py`	The starter workflows (ADWs), e.g. `adw_scout`, `adw_plan_build_test`, `adw_simple_sdlc`
`adws/adw_modules/`	The factory's own code. Protected: agents may not edit it.
`adws/adw_sssf_config/sssf.config.yaml`	The agent roster: models, tools, write permissions, timeouts
`adws/adw_data/prompt_engineering/`	Each agent's prompts. Yours to edit.
`adws/adw_data/harness_engineering/`	Pi extensions and the Context7 MCP config
`justfile`, `.env.sample`	Recipes and the environment template
`.gitignore` entries	Runtime data (`adws/adw_data/sessions/`, `sssf.db`), `.env`, bytecode

Re-running it is safe: existing files are skipped and listed, so a second run doubles as a drift check.

6 Set your API keys

LINUX / MACOS / GIT BASH

```
cp .env.sample .env
```

WINDOWS POWERSHELL

```
Copy-Item .env.sample .env
```

Then fill in `.env`. Which keys you need depends on the `model:` lines in `sssf.config.yaml`: each is written `provider/model-id`, and the provider decides the key. The starter roster deliberately mixes three providers:

Starter model	Used by	Key
`google/gemini-3.6-flash` (via OpenRouter)	default, builder, scout	`OPENROUTER_API_KEY`
`fireworks/…/kimi-k3`	planner	`FIREWORKS_API_KEY`
`openai/gpt-5.6-terra`, `openai/gpt-5.6-luna`	reviewer, documenter	`OPENAI_API_KEY`

> **One key instead of three:** delete the per-agent `model:` lines so every agent inherits `defaults.model`. A missing key is not caught at startup; it fails when that agent runs.

Context7 (library docs) needs no key; `CONTEXT7_API_KEY` only raises its rate limit. The docs phase sends library names, versions and the first 300 characters of your request to Context7. Set

`context7.send_request: false` to send names only, or `context7.enabled: false` to turn it off.

7 Make the first commit

Workflows that commit need at least one commit in the repository. Commit the installed factory (`.env` is gitignored, so your keys stay out):

```
git add -A
git commit -m "Install sssf"
```

Start runs from a clean tree. A commit phase only commits what the run changed and leaves your own uncommitted files out, but a clean tree avoids the question.

8 Wire your real test command

> **Do this before trusting any test workflow.** `adws/adw_modules/quality.py` ships with placeholder commands. On Linux and macOS they print a notice and pass, so a fresh install reports a green test phase that tested nothing.

Open `adws/adw_modules/quality.py` and replace each `_placeholder(...)` with the real command, as a list:

```
argv=["uv", "run", "pytest", "-q"]      # or ["bun", "test"], ["npm", "run", "lint"], …
```

Delete the blocks you don't need and remove them from `run_quality()`.

9 Smoke test

```
just demo        # two cheap, read-only runs, end to end
just sessions    # what just happened
just obs         # the trace UI at http://localhost:4601 (needs bun)
```

Without `just`, run the first half of the demo directly:

```
uv run adws/adw_prompt.py "reply with a one-line summary of this repo" --agent scout
```

Success means the config validated, an agent ran, its report parsed, and the trace landed in `adws/adw_data/sssf.db`. Fix any failure here before running larger workflows: every multi-agent workflow rides this same path. The trace UI needs that database, so run the demo before `just obs`.

10 Windows notes

The justfile runs on Git for Windows' shell

The stamped `justfile` starts with:

```
set windows-shell := ["C:/Program Files/Git/bin/sh.exe", "-cu"]
```

On Windows, `just` otherwise runs recipes with PowerShell or `cmd`, which do not understand the POSIX syntax the recipes use. This line makes every recipe run in Git for Windows' `sh`, so the recipes behave the same on every OS. Linux and macOS ignore it.

- **Git installed somewhere else** (for example per-user under `%LOCALAPPDATA%\Programs\Git`)? Edit the path in that line. Find it with `where git`: the shell is `bin\sh.exe` next to the `cmd` folder that output names.
- Use forward slashes in the path, as shown, or doubled backslashes.

Running the commands

- `uv run …`, `git …` and `just …` commands work unchanged in PowerShell.
- Windows PowerShell 5 has no `&&`; run chained commands one per line, as this guide does.
- All files and agent output are read and written as UTF-8 regardless of the Windows code page, so non-English text and emoji in prompts or fetched docs are safe.

Behaviour that differs on Windows

- **Stopping a run.** On Linux and macOS, killing an ADW (or an agent timing out) also kills every process the agent started. On Windows only the agent process itself is terminated; a test server it launched may need closing by hand.
- **Placeholder test commands** may fail with exit 127 instead of passing, because `echo` is not a program on Windows unless Git's Unix tools are on PATH. Either way, step 8 replaces them.

11 Updating an installation

The installer never overwrites a file that exists, so a new skill version is applied in a few steps. Do it when no run is in progress, from the project root, after committing everything (`git add -A`, `git commit -m "Before sssf update"`).

LINUX / MACOS / GIT BASH

```
# 1. latest skill
git -C ~/tools/ssaf pull
rm -rf .claude/skills/sssf
cp -r ~/tools/ssaf/.claude/skills/sssf .claude/skills/
# 2. new files only
uv run .claude/skills/sssf/scripts/install.py
# 3. the factory's code
T=.claude/skills/sssf/templates/adws
cp -r $T/adw_modules/. adws/adw_modules/
cp $T/adw_*.py $T/conftest.py adws/
```

WINDOWS POWERSHELL

```
# 1. latest skill
git -C C:\tools\ssaf pull
Remove-Item -Recurse -Force .claude\skills\sssf
Copy-Item -Recurse C:\tools\ssaf\.claude\skills\sssf .claude\skills\
# 2. new files only
uv run .claude/skills/sssf/scripts/install.py
# 3. the factory's code
$T = ".claude\skills\sssf\templates\adws"
Copy-Item -Recurse -Force $T\adw_modules\* adws\adw_modules\
Copy-Item -Force "$T\adw_*.py", "$T\conftest.py" adws\
```

4. **Put your test commands back.** Step 3 replaced `quality.py` too. Run `git diff adws/adw_modules/quality.py` and copy your removed `argv=…` lines back in place of `_placeholder(…)`.
5. **Compare the files you own** with their templates and merge what you want; old configs keep working, because new settings have defaults:
   `git diff --no-index .claude/skills/sssf/templates/sssf.config.yaml adws/adw_sssf_config/sssf.config.yaml`
   `git diff --no-index .claude/skills/sssf/templates/justfile justfile`
6. **Check and commit:** `just demo`, then `git add -A` and `git commit -m "Update sssf"`.

> Avoid `install.py --force` for updates: it overwrites **all** stamped files, including your config, your prompts and `quality.py`. Its only exception is a `justfile` that did not come from sssf.

12 Troubleshooting

Symptom	Cause	Fix
`/sssf` is unknown in Claude Code	The skill is not at `.claude/skills/sssf/SKILL.md` under the folder Claude Code was opened in	Step 4: check the layout, and open Claude Code in the project root
`adws/` appeared in the wrong folder	The installer was run from somewhere other than the project root	Delete what it created there, `cd` to the root (step 3), run it again
Installer says “stamped: 0” after an update	Normal when nothing is new; if you expected new files, the skill copy is old	`git -C ~/tools/ssaf pull`, then redo step 1 of “Updating”
`config validation failed` before anything runs	A model name is ambiguous or unknown, or a prompt file is missing	Write models as `provider/model-id`; check `pi --list-models`
An agent fails partway through a workflow	Its provider's API key is not set	Add the key to `.env` (step 6)
A commit phase says “not a git repository” or “nothing to commit”	No repository, or the run changed nothing	Steps 3 and 7

Test phase is green but nothing was tested	Placeholder commands in `quality.py`	Step 8
`just obs` says `sssf.db not found`	No run has happened yet	Run `just demo` first
On Windows, `just` cannot find `sh.exe`	Git for Windows is missing or not at the default path	Install it, or edit the `windows-shell` line (step 10)
A run seems stuck	An agent stopped responding	`just procs <adw_id>` shows its processes; each agent call is also killed after `timeout_s` (default one hour, set in `sssf.config.yaml`)
The docs phase reports errors	Context7 unreachable or rate-limited; the run continues without docs	Allow `mcp.context7.com` on your network, or set `CONTEXT7_API_KEY`
An agent's change was rolled back and the phase failed	It wrote outside its `writes:` permission	Expected. Widen that agent's `writes:` if the change was legitimate

More detail: the repository README and `.claude/skills/sssf/references/config.md`.